

Universitatea
Transilvania
din Brașov

FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

Optimizarea filtrului LMS pentru reducerea zgomotului vocal pe Raspberry Pi 5 utilizând extensii SIMD

Nume complet: Solomon Vlad-George

BRAȘOV, 2025

1. Rezumat

Acest proiect urmărește implementarea și testarea unui algoritm de filtrare adaptivă LMS aplicat unui semnal vocal contaminat cu zgomot alb, folosind platforma Raspberry Pi 5. Au fost evaluate performanțele algoritmului în versiune neoptimizată și optimizată prin extensii SIMD specifice arhitecturii ARMv8. De asemenea, s-a analizat influența compandării μ -law asupra calității filtrării. Performanța a fost evaluată prin indicatori precum SNR, MSE și timpul de execuție, în funcție de parametrii μ , M și nivelul de zgomot σ .

2. Introducere

Zgomotul aditiv, de tip alb, reprezintă una dintre cele mai comune surse de degradare în procesarea semnalelor vocale, afectând inteligibilitatea și claritatea percepută. Filtrarea adaptivă [1][6] oferă o soluție eficientă pentru astfel de perturbări, permițând ajustarea continuă a răspunsului în funcție de semnalul de intrare. Algoritmul LMS (Least Mean Squares) este frecvent utilizat în acest scop datorită simplității implementării și capacității sale de adaptare.[1][4]

În cadrul acestui proiect se propune implementarea unei soluții software de filtrare adaptivă a semnalelor audio folosind platforma Raspberry Pi 5[5], cu accent pe optimizarea execuției prin vectorizare hardware (extensii SIMD NEON)[3]. Se urmărește, de asemenea, evaluarea influenței compandării μ -law asupra comportamentului filtrului. Testarea este realizată pe înregistrări vocale contaminate artificial cu zgomot, fiind analizate atât performanțele de curățare (SNR, MSE), cât și eficiența computațională.

3. FUNDAMENTE TEORETICE

Algoritmul LMS (Least Mean Squares)

Algoritmul LMS este o metodă adaptivă de filtrare ce ajustează coeficienții filtrului în timp real pentru a minimiza eroarea dintre un semnal dorit și semnalul estimat. Este utilizat frecvent în aplicații de reducere a zgomotului, ecoului și compensare de canal. Actualizarea coeficienților se face cu regula:

$$w_i(n+1) = w_i(n) + 2 \cdot \mu \cdot e(n) \cdot x(n-i) [1]$$

- w_i sunt coeficienții filtrului,
- μ este rata de învățare,

- $e(n)$ este eroarea între semnalul dorit și cel estimat,
- $x(n - i)$ este semnalul de intrare întârziat.

În cadrul proiectului, au fost analizate performanțele algoritmului pentru mai multe valori ale parametrilor μ și M (numărul de coeficienți ai filtrului).

Estimarea ieșirii: $\hat{y} = \sum_{i=0}^{M-1} w[i] * x[n - i]$

Eroarea: $e[n] = d[n] - \hat{y}[n]$

Formula standard LMS (actualizarea): $w[i] = w[i] + 2 \cdot \mu \cdot e[n] \cdot x[n - i]$

Compandarea μ -law

Compandarea μ -law este o tehnică nelineară folosită pentru a comprima intervalul dinamic al semnalelor audio, în special în comunicațiile vocale. Scopul este de a păstra detaliile semnalului la amplitudini mici și de a reduce impactul zgomotului pe cele mari.

Formula de compresie: $y = \text{sign}(x) \frac{\log(1+\mu|x|)}{\log(1+\mu)}$ [7]

și cea de expansiune: $x = \text{sign}(y) \frac{(1+\mu)^{|y|}-1}{\mu}$ unde $\mu = 255$, iar $x \in [-1,1]$. [7]

În proiect, compandarea a fost aplicată înainte de filtrare, iar expansiunea a fost utilizată ulterior pentru comparație.

Zgomotul alb

Zgomotul alb este un semnal aleatoriu cu densitate spectrală constantă, adică are aceeași energie pe toate frecvențele. În simulări, acest tip de zgomot este ideal pentru a testa robustețea filtrului adaptiv. Nivelul de zgomot a fost controlat prin parametrul σ , indicând amplitudinea acestuia.

Măsuri de performanță – SNR și MSE

SNR (Signal-to-Noise Ratio): măsoară cât de clar este semnalul filtrat în raport cu zgomotul rămas. Se calculează cu:

$$SNR = 10 \cdot \log_{10} \frac{P_{\text{semnal}}}{P_{\text{eroare}}} [2]$$

MSE (Mean Squared Error): exprimă eroarea medie pătratică dintre semnalul curat și cel estimat:

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2 \quad [2]$$

Optimizarea cu SIMD (NEON – ARMv8)

Instrucțiunile SIMD (Single Instruction, Multiple Data) permit executarea în paralel a acelorași operații asupra mai multor date, crescând semnificativ performanța pentru calcule vectoriale, cum sunt cele din algoritmul LMS.

4. Implementare

Implementarea proiectului a fost realizată în limbajul C, rulând nativ pe platforma Raspberry Pi 5, un microcomputer echipat cu procesor ARM Cortex-A76[1]. Scopul a fost simularea unui sistem de filtrare adaptivă pentru semnale vocale afectate de zgomot alb, folosind algoritmul LMS (Least Mean Squares) în două variante: una de bază, și una care include compandare de tip μ -law.[7]

Structura codului este centrată pe următoarea succesiune logică: inițial, fișierul audio curat (voice_clean.wav) este citit în memorie folosind biblioteca libsndfile. Ulterior, peste semnalul original se adaugă zgomot alb artificial, controlat de un parametru σ , care definește intensitatea acestuia. Pentru fiecare nivel de zgomot testat, se parcurg toți parametrii μ (rata de învâțare) și M (dimensiunea filtrului), aplicați asupra algoritmului LMS.[1]

În varianta cu compandare activă, semnalul este prelucrat înainte de filtrare printr-o funcție nelineară specifică standardului μ -law, utilizată adesea în compresia vocală. Atât semnalul curat cât și zgomotul sunt compandate separat, filtrul LMS este aplicat în domeniul compresat, iar rezultatul este expandat înapoi în domeniul original pentru a putea fi comparat corect cu semnalul de referință.

La fiecare rulare, sunt calculate automat performanțele algoritmului: SNR (Signal-to-Noise Ratio), MSE (Mean Squared Error) și timpul de execuție[2][4], care sunt salvate într-un fișier .csv. În plus, pentru fiecare configurație testată, este generat un fișier audio filtrat (voice_filtered_*.wav) care permite evaluarea auditivă a rezultatului.

Codul a fost compilat cu optimizări specifice arhitecturii ARMv8 și a extensiilor SIMD (NEON), folosind opțiuni de tipul -O3, -march=armv8-a+simd și -ftree-vectorize, ceea ce a permis obținerea unei vectorizări eficiente la nivel de compilator[3]. Aceste optimizări s-au reflectat într-o reducere semnificativă a timpului de execuție, varianta optimizată fiind de până la 12 ori mai rapidă decât cea de referință neoptimizată.

Astfel, implementarea propusă permite testarea și compararea flexibilă a mai multor configurații de filtrare, oferind atât evaluări numerice, cât și posibilitatea de validare auditivă, totul într-un cadru complet automatizat și portabil.

5. Teste si experimente

Pentru a evalua performanța algoritmului LMS implementat pe platforma Raspberry Pi 5, au fost realizate o serie de teste care vizează atât acuratețea filtrării, cât și eficiența execuției. Au fost comparate două versiuni ale codului: una optimizată cu vectorizare SIMD (prin opțiunile de compilare ARM NEON) și una neoptimizată. Testele s-au realizat pe un fișier audio contaminat cu zgomot alb de diferite intensități (σ), urmărind metrice precum timpul de execuție, raportul semnal-zgomot (SNR), eroarea medie pătratică (MSE) și spectrul rezultat.

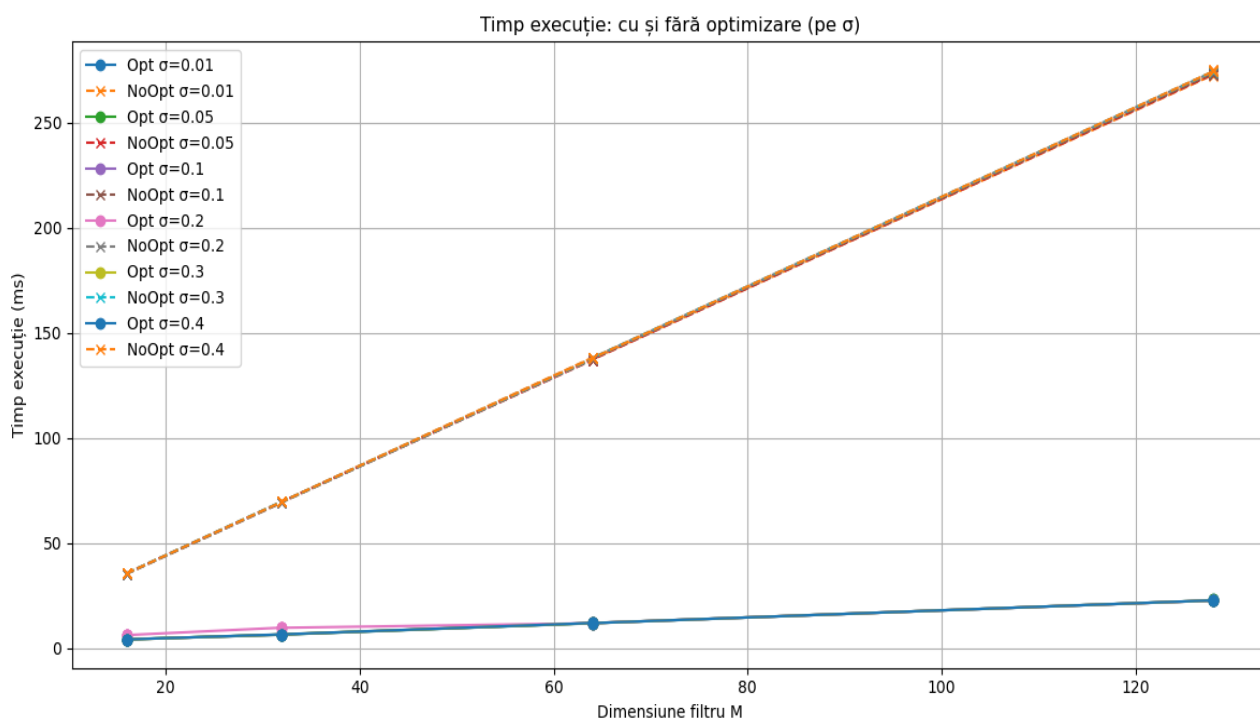


Fig.1 Comparație timp de execuție versiunea optimizată/neoptimizată

Graficul evidențiază impactul semnificativ al optimizărilor (vectorizare SIMD) asupra timpului de execuție. Versiunea optimizată scade timpul cu un ordin de mărime, în special pentru filtre mai lungi. Versiunea neoptimizată prezintă o creștere liniară accentuată a timpului.

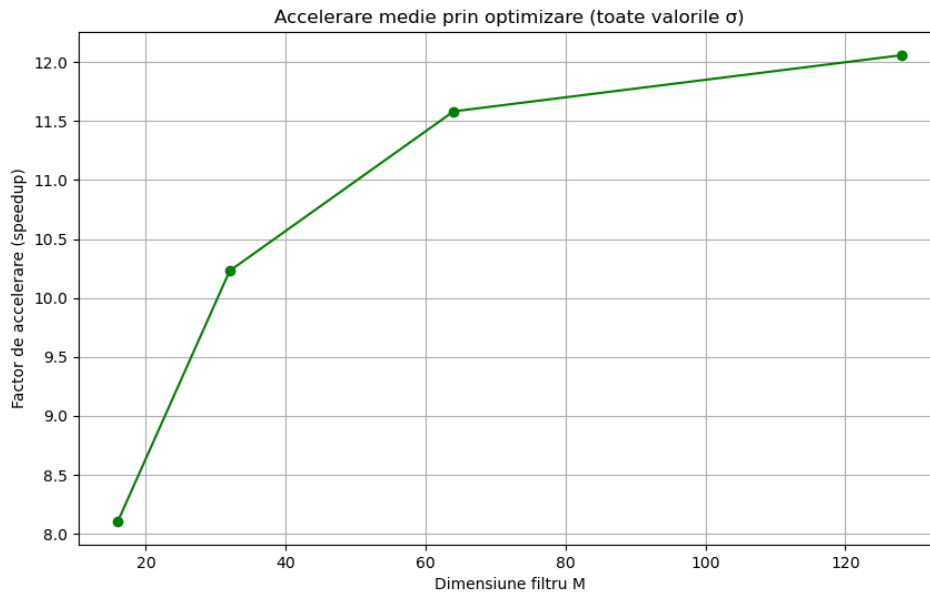


Fig 2. Factorul de accelerare (speedup) mediu obținut prin optimizare pentru fiecare dimensiune M

Se observă o accelerare consistentă, de aproximativ 8–12×, prin utilizarea extensiilor SIMD. Câștigul crește odată cu dimensiunea filtrului, ceea ce validează eficiența vectorizării în prelucrări de semnal intensive.

Pentru comparația performanței între codul optimizat și cel neoptimizat, s-au rulat aceleași configurații ale algoritmului LMS asupra semnalului contaminat, iar timpul de execuție a fost măsurat folosind funcțiile `clock()` din C. Rezultatele au fost salvate în două fișiere .csv separate (`rezultate_opt.csv` pentru varianta optimizată și `rezultate_noopt.csv` pentru cea neoptimizată). Graficele au fost generate cu ajutorul scripturilor Python, prin agregarea timpilor în funcție de dimensiunea filtrului M, pentru toate valorile de zgomot σ , precum și prin calcularea factorului mediu de accelerare (speedup).

În figura 3, graficul prezintă distribuția spectrală a celor trei semnale testate: semnalul original (albastru), semnalul cu zgomot alb adăugat (portocaliu) și cel rezultat după filtrarea LMS (verde), pentru $\sigma=0.4$.

Se observă că vocea curată are energie concentrată în zona 0–1000 Hz, specifică formantelor vocale. După contaminarea cu zgomot alb, spectrul devine aproape plat, cu amplitudine crescută pe toată banda. În urma filtrării, curba verde arată că LMS reușește să atenueze semnificativ zgomotul în zona de frecvență joasă, unde se află semnalul util. După 1500–2000 Hz, semnalul filtrat este mult estompat, dar acest lucru nu afectează semnificativ percepția auditivă, deoarece informația vocală esențială rămâne concentrată în partea inferioară a spectrului.

Prin urmare, chiar dacă filtrul nu recuperează complet spectrul original, SNR-ul este relativ bun tocmai pentru că reduce eroarea exact acolo unde vocea este predominantă.

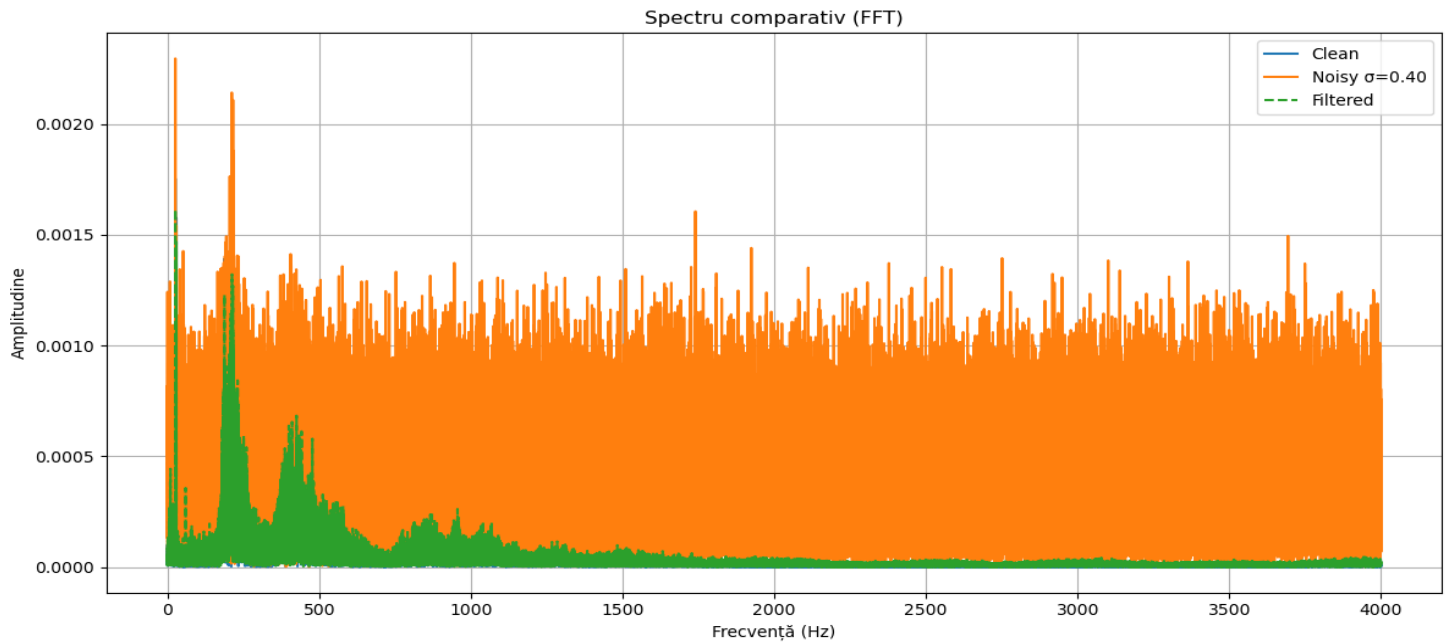


Fig. 3 Spectru comparativ pentru semnal curat, cu zgomot ($\sigma = 0.4$) și filtrat cu LMS

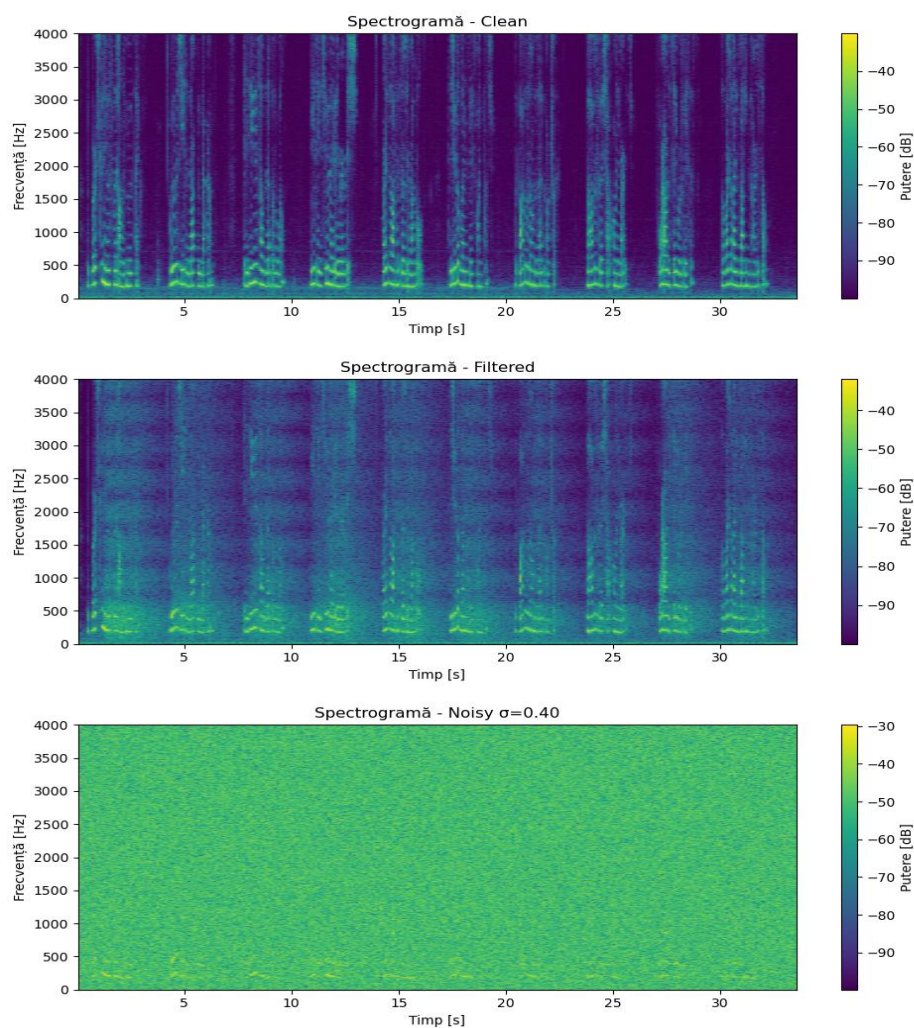


Fig.4 Spectrograme comparative pentru semnalul curat, filtrat și zgomotos ($\sigma = 0.4$)

În prima bandă (sus), semnalul curat are energie bine localizată în benzi de frecvență joasă și medie (sub 3–4 kHz), cu o structură verticală specifică vocalelor și consoanelor, clar delimitată în timp.

În partea de jos (zgomotul), spectrograma este uniformă pe toată durata și gama de frecvențe — o caracteristică tipică zgomotului alb. Se remarcă lipsa oricărei structurări temporale sau spectrale.

Banda din mijloc, corespunzătoare semnalului filtrat, arată o recuperare parțială a structurilor inițiale din semnalul curat. Formanții vocali principali (componentele consistente în frecvență) sunt din nou vizibili în partea inferioară a spectrului. Deși se mai observă urme de zgomot rezidual, în special în zonele înalte, filtrul reușește să elimine zgomotul masiv și să păstreze inteligibilitatea vocii.

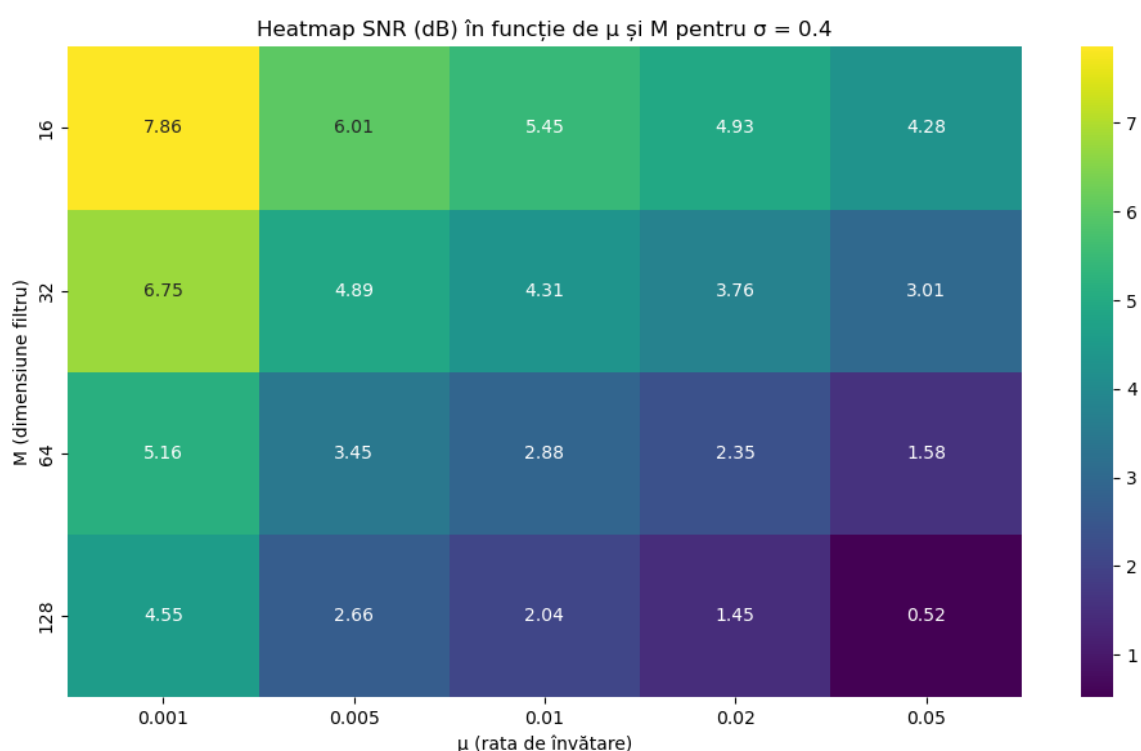


Fig 5. Heatmap SNR (dB) în funcție de μ și M pentru $\sigma = 0.4$

Această imagine arată variația raportului semnal-zgomot (SNR) în funcție de doi parametri importanți ai filtrului LMS: dimensiunea filtrului (M) și rata de învățare (μ), pentru un nivel ridicat de zgomot ($\sigma = 0.4$). Se observă clar că valorile cele mai mari ale SNR apar atunci când M este mic (16 sau 32) și μ are o valoare redusă (de exemplu 0.001).

Acest comportament este justificat de natura zgomotului alb — lipsit de corelații temporale — ceea ce înseamnă că un filtru prea lung introduce coeficienți inutili, care afectează stabilitatea și performanța. În plus, un filtru LMS de dimensiune mică se adaptează mai rapid și mai stabil, ceea ce îl face mai eficient în condiții de zgomot ridicat. Creșterea dimensiunii M fără

ajustarea corespunzătoare a μ duce la scăderea performanței, așa cum indică scăderea progresivă a SNR în heatmap.

ormula de actualizare a coeficienților este: $w(n+1) = w(n) + 2\mu e(n)x(n)$

Așadar, odată cu creșterea lui M , numărul de coeficienți de adaptat crește liniar, iar rata de convergență scade, necesitând mai mult timp (și date) pentru stabilizare.

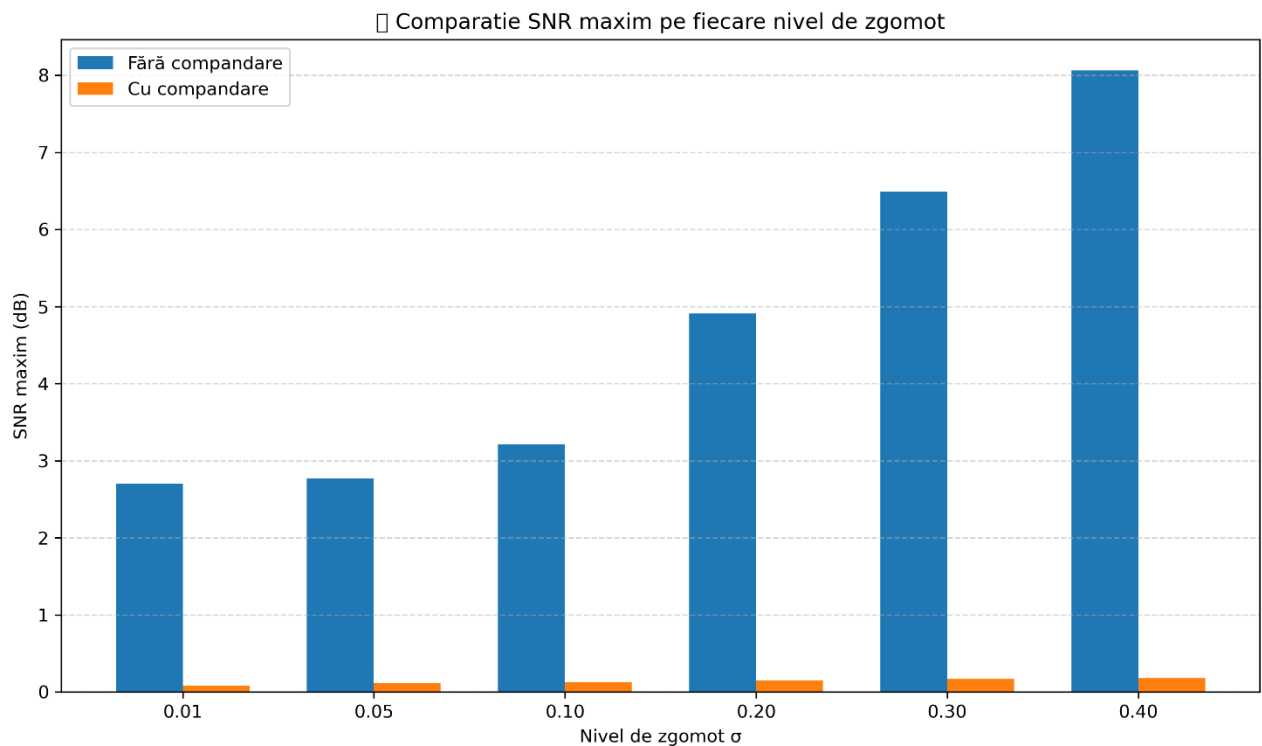


Fig.6 Compararea valorilor maxime ale SNR pentru fiecare nivel de zgomot (cu și fără compandare)

În cazul fără compandare, SNR-ul crește odată cu zgomotul: filtrul LMS are mai multe componente „de învățat” și, astfel, poate corecta mai bine zgomotul, crescând contrastul între semnal și eroare.

În cazul cu compandare, valorile SNR sunt constant mult mai mici, pentru că filtrul LMS nu poate estima corect distorsiunile introduse de funcția neliniară de compresie (μ -law). Compandarea modifică distribuția semnalului într-un mod greu de prezis pentru un filtru liniar adaptiv.

Compandarea μ -law este concepută pentru a îmbunătăți raportul semnal-zgomot în sisteme de comunicații cu semnale slabe sau nonuniform distribuite (de exemplu, voce naturală transmisă prin canale cu zgomot aditiv *non-white*). Însă în cazul de față:

S-a adăugat zgomot alb, adică zgomot uniform distribuit pe toate frecvențele și amplitudinile.

Compandarea acționează neliniar asupra acestui zgomot, ceea ce deformează distribuția semnalului într-un mod imprevizibil pentru filtrul LMS.

Rezultatul este o creștere a erorii de estimare și implicit o scădere drastică a SNR-ului (vizibil în barele portocalii din grafic).

6. Concluzii

În urma testelor efectuate, s-a constatat că algoritmul LMS aplicat pe semnale vocale contaminate cu zgomot alb oferă rezultate bune atunci când sunt alese dimensiuni mici ale filtrului ($M = 16-32$) și o rată de învățare scăzută ($\mu = 0.001$). Filtrul reușește să păstreze componentele vocale esențiale din zona joasă de frecvență (sub 2 kHz), unde este concentrată energia utilă a semnalului.

Optimizarea codului C pentru arhitectura ARMv8 cu extensii SIMD (NEON) a condus la o accelerare de până la 12× în timp de execuție, fără a afecta calitatea filtrării. Diferențele au fost evidențiate prin comparații directe între timpii de rulare și prin analiza valorilor SNR.

Testele cu compandare μ -law au arătat că, în cazul zgomotului alb, această tehnică reduce eficiența LMS-ului, deoarece distorsionează semnalul într-un mod greu de estimat de un filtru liniar adaptiv. Astfel, compandarea s-a dovedit ineficientă în acest context.

7. Bibliografie

- [1] S. Haykin, Adaptive Filter Theory, 5th ed., Pearson, 2013.
- [2] R. C. Gonzalez and R. E. Woods, Digital Signal Processing, Pearson, 2008.
- [3] ARM Ltd., CMSIS-DSP Software Library User Guide, https://arm-software.github.io/CMSIS_5/DSP/html/index.html
- [4] MathWorks, "What is the LMS adaptive filter?", <https://www.mathworks.com/help/dsp/ref/dsp.lmsfilter-system-object.html>
- [5] Raspberry Pi Ltd., Raspberry Pi 5 Technical Specifications, <https://www.raspberrypi.com/products/raspberry-pi-5/>
- [6] N. Dey, A. Ashour, and S. Balas, Applied Speech Processing: Adaptation and Optimization, Springer, 2021.
- [7] ITU-T Recommendation G.711: Pulse code modulation (PCM) of voice frequencies, International Telecommunication Union, 1988.